Midterm HTTP Server Project

Generated by Doxygen 1.17.0

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Chapter 3

Data Structure Documentation

3.1 client_info Struct Reference

Stores state and metadata for an active client connection.

`#include <network_utils.h>`

Data Fields

- int address_length
  The actual size of the stored address.
- struct sockaddr_storage address
  The binary IP/Port address of the client.
- char address_buffer [128]
  A human-readable string of the client's IP.
- SOCKET socket
  The active socket descriptor for this client.
- char request [MAX_REQUEST_SIZE+1]
  Buffer containing the incoming HTTP request.
- int received
  Current number of bytes stored in the request buffer.
- struct client_info * next
  Pointer to the next client in the linked list.

3.1.1 Detailed Description

Stores state and metadata for an active client connection.

3.1.2 Field Documentation

3.1.2.1 address

`struct sockaddr_storage client_info::address`

The binary IP/Port address of the client.

3.1.2.2 address_buffer

```
char client_info::address_buffer[128]
```

A human-readable string of the client's IP.

3.1.2.3 address_length

```
int client_info::address_length
```

The actual size of the stored address.

3.1.2.4 next

```
struct client_info* client_info::next
```

Pointer to the next client in the linked list.

3.1.2.5 received

```
int client_info::received
```

Current number of bytes stored in the request buffer.

3.1.2.6 request

```
char client_info::request[MAX_REQUEST_SIZE+1]
```

Buffer containing the incoming HTTP request.

3.1.2.7 socket

```
SOCKET client_info::socket
```

The active socket descriptor for this client.

The documentation for this struct was generated from the following file:

- network_utils.h

Chapter 4

File Documentation

4.1 debug_utils.h File Reference

Debugging Helpers.

```
#include "winhders.h"
#include <stdio.h>
```

Macros

- #define DEBUG_LOG(fmt, ...)
- #define LAST_SOCKET_ERROR()

4.1.1 Detailed Description

Debugging Helpers.

Author

David A. Sowles

Version

Midterm

4.1.2 Macro Definition Documentation

4.1.2.1 DEBUG_LOG

```
#define DEBUG_LOG(
              fmt,
              ...)
```

4.1.2.2 LAST_SOCKET_ERROR

```
#define LAST_SOCKET_ERROR()
```

4.2 debug_utils.h

Go to the documentation of this file.

```
00001
00007
00008 #pragma once
00009
00010 #include "winhders.h"
00011 #include <stdio.h>
00012
00013 //------------------------------------------------------------------------------
00014 // Definitions
00015 //------------------------------------------------------------------------------
00016
00017 /*************************************************
00018  * MAKE SURE TO COMMENT THIS OUT FOR RELEASE BUILDS *
00019  *************************************************/
00020 //#define DEBUG
00021 /************************************************* */
00022
00023 #ifdef DEBUG
00027     #define DEBUG_LOG(fmt, ...) \
00028     fprintf(stderr, "DEBUG [%s:%d]: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)
00029
00031     #define LAST_SOCKET_ERROR() \
00032     fprintf(stderr, "DEBUG [%s:%d]: Last Winsock error: %d\n", __FILE__, __LINE__, WSAGetLastError())
00033
00034 #else
00035     // Debug logging is disabled in release builds
00036     #define DEBUG_LOG(fmt, ...) // Does nothing in release builds
00037
00038     #define LAST_SOCKET_ERROR()
00039 #endif
00040
00041 //------------------------------------------------------------------------------
```

4.3 http_server.c File Reference

The server code file for the server.

```
#include "debug_utils.h"
#include "http_server.h"
#include "network_utils.h"
```

Functions

- int main ()

4.3.1 Detailed Description

The server code file for the server.

Author

David A. Sowles

Version

Midterm

4.3.2 Function Documentation

4.3.2.1 main()

```
int main ()
```

4.4 http_server.h File Reference

An HTTP Server.

```
#include "winhders.h"
#include <stdio.h>
#include <stdlib.h>
```

Macros

- #define DEFAULT_PORT "8080"

  The default port the server listens on if none is specified.
- #define BUFFER_SIZE 4096

  Size of the temporary buffer for reading/writing file chunks.

Variables

- WORD REQUIRED_WINSOCK_VERSION = MAKEWORD(2,2)

  The Winsock version preferred by the server app.

4.4.1 Detailed Description

An HTTP Server.

Author

David A. Sowles

Version

Midterm

4.4.2 Macro Definition Documentation

4.4.2.1 BUFFER_SIZE

```
#define BUFFER_SIZE 4096
```

Size of the temporary buffer for reading/writing file chunks.

4.4.2.2 DEFAULT_PORT

```
#define DEFAULT_PORT "8080"
```

The default port the server listens on if none is specified.

4.4.3 Variable Documentation

4.4.3.1 REQUIRED_WINSOCK_VERSION

```
WORD REQUIRED_WINSOCK_VERSION = MAKEWORD(2,2)
```

The Winsock version preferred by the server app.

4.5 http_server.h

Go to the documentation of this file.

```
00001
00007
00008 #pragma once
00009
00010 #include "winhders.h"
00011 #include <stdio.h>
00012 #include <stdlib.h>
00013
00014
00015 //-------------------------------------------------------------------------------
00016 // Definitions
00017 //-------------------------------------------------------------------------------
00018
00020 #define DEFAULT_PORT "8080"
00021
00023 #define BUFFER_SIZE 4096
00024
00026 WORD REQUIRED_WINSOCK_VERSION = MAKEWORD(2,2);
00027
00028 //-------------------------------------------------------------------------------
```

4.6 network_utils.c File Reference

```
#include "network_utils.h"
#include <stdlib.h>
#include "debug_utils.h"
```

Macros

- #define BSIZE 1024

Functions

- void start_winsock (WORD wVersion, LPWSADATA lpWSADATA)

  Starts the Winsock API for the app.
- const char * get_content_type (const char *path)

  Returns the content-type value appropriate for the path arg's extension type.
- SOCKET create_listening_socket (const char *host, const char *port)

  Creates and binds a socket for a server to listen on.
- void print_socket_info (const SOCKET sock)

  Prints information about a network socket, given a bare socket as input.
- struct client_info * get_client_info (struct client_info **client_info_list, SOCKET sock)

  Searches the linked list for a client associated with a specific socket.
- void drop_client (struct client_info **client_info_list, struct client_info *client_info)

  Removes the given client from the server's client list of connections. Also frees memory used by client.
- const char * get_client_address (struct client_info *info)

  Retrieves the human-readable IP address string for a client.
- fd_set wait_for_network_event (struct client_info **client_info_list, SOCKET listen_sock)

  Waits until a client's socket is ready to be handled.
- void send_400 (struct client_info **client_info_list, struct client_info *client_info)

  Sends an HTTP 400 Bad Request response and closes the connection.
- void send_404 (struct client_info **client_info_list, struct client_info *client_info)

  Sends an HTTP 404 Not Found response and closes the connection.
- void serve_resource (struct client_info **client_info_list, struct client_info *client_info, const char *path)

  Locates a requested file on disk and transmits it to the client.

4.6.1 Macro Definition Documentation

4.6.1.1 BSIZE

```
#define BSIZE 1024
```

4.6.2 Function Documentation

4.6.2.1 create_listening_socket()

```
SOCKET create_listening_socket (
            const char * host,
            const char * port)
```

Creates and binds a socket for a server to listen on.

Parameters

host	The local address to bind to (use NULL or 0 for any available interface).
port	The port number or service name (e.g., "8080").

Returns

A bound socket ready for listening.

4.6.2.2 drop_client()

```
void drop_client (
            struct client_info ** client_list,
            struct client_info * client)
```

Removes the given client from the server's client list of connections. Also frees memory used by client.

Parameters

client_list	A pointer to the head item in the client linked-list.
client	The client connection to be removed and shutdown.

4.6.2.3 get_client_address()

```
const char * get_client_address (
            struct client_info * info)
```

Retrieves the human-readable IP address string for a client.

Parameters

info	A pointer to the client_info struct.

Returns

A constant pointer to the client's IP address string.

4.6.2.4 get_client_info()

```
struct client_info * get_client_info (
            struct client_info ** client_list,
            SOCKET sock)
```

Searches the linked list for a client associated with a specific socket.

Parameters

client_list	A pointer to the head of the client linked list.
sock	The socket descriptor to search for.

Returns

A pointer to the client_info struct if found; NULL otherwise.

4.6.2.5 get_content_type()

```
const char * get_content_type (
            const char * path)
```

Returns the content-type value appropriate for the path arg's extension type.

Parameters

path	A valid path string to some file.

Returns

The appropriate content-header value for the path's extension.

4.6.2.6 print_socket_info()

```
void print_socket_info (
            const SOCKET sock)
```

Prints information about a network socket, given a bare socket as input.

Parameters

sock	The socket.

4.6.2.7 send_400()

```
void send_400 (
            struct client_info ** client_list,
            struct client_info * client)
```

Sends an HTTP 400 Bad Request response and closes the connection.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client that sent the malformed request.

4.6.2.8 send_404()

```
void send_404 (
            struct client_info ** client_list,
            struct client_info * client)
```

Sends an HTTP 404 Not Found response and closes the connection.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client that requested a non-existent resource.

4.6.2.9 serve_resource()

```
void serve_resource (
            struct client_info ** client_list,
            struct client_info * client,
            const char * path)
```

Locates a requested file on disk and transmits it to the client.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client requesting the resource.
path	The local filesystem path to the requested file.

4.6.2.10 start_winsock()

```
void start_winsock (
            WORD wVersion,
            LPWSADATA lpWSADATA)
```

Starts the Winsock API for the app.

Parameters

wVersion	The version of Winsock to be requested.
lpWSADATA	A structure holding the Winsock module's information.

4.6.2.11 wait_for_network_event()

```
fd_set wait_for_network_event (
            struct client_info ** client_list,
            SOCKET server)
```

Waits until a client's socket is ready to be handled.

Parameters

client_list	A pointer to the head of the connected clients list.
server	The listening server socket to monitor for new connections.

Returns

An fd_set containing all sockets that are ready for an I/O operation.

4.7 network_utils.h File Reference

A set of network utility functions for the server.

```
#include "winhders.h"
```

Data Structures

- struct client_info

  Stores state and metadata for an active client connection.

Macros

- #define MAX_REQUEST_SIZE 2047

  The maximum allowable size for an incoming HTTP request.
- #define STATUS_LOG(fmt, ...)

  A basic macro function for printing status messages to stdout.

Functions

- void start_winsock (WORD wVersion, LPWSADATA lpWSADATA)

  Starts the Winsock API for the app.
- const char * get_content_type (const char *path)

  Returns the content-type value appropriate for the path arg's extension type.
- SOCKET create_listening_socket (const char *host, const char *port)

  Creates and binds a socket for a server to listen on.
- void print_socket_info (const SOCKET sock)

  Prints information about a network socket, given a bare socket as input.
- struct client_info * get_client_info (struct client_info **client_list, SOCKET sock)

  Searches the linked list for a client associated with a specific socket.

- void drop_client (struct client_info **client_list, struct client_info *client)

  Removes the given client from the server's client list of connections. Also frees memory used by client.
- const char * get_client_address (struct client_info *info)

  Retrieves the human-readable IP address string for a client.
- fd_set wait_for_network_event (struct client_info **client_list, SOCKET server)

  Waits until a client's socket is ready to be handled.
- void send_400 (struct client_info **client_list, struct client_info *client)

  Sends an HTTP 400 Bad Request response and closes the connection.
- void send_404 (struct client_info **client_list, struct client_info *client)

  Sends an HTTP 404 Not Found response and closes the connection.
- void serve_resource (struct client_info **client_list, struct client_info *client, const char *path)

  Locates a requested file on disk and transmits it to the client.

Variables

- WORD winsock_version

4.7.1 Detailed Description

A set of network utility functions for the server.

Author

David A. Sowles

Version

Midterm

4.7.2 Macro Definition Documentation

4.7.2.1 MAX_REQUEST_SIZE

```
#define MAX_REQUEST_SIZE 2047
```

The maximum allowable size for an incoming HTTP request.

4.7.2.2 STATUS_LOG

```
#define STATUS_LOG(
                fmt,
                ...)
```

Value:

```
    do { \
    fprintf(stdout, fmt "\n" __VA_OPT__(,) __VA_ARGS__); \
    fflush(stdout); \
} while(0)
```

A basic macro function for printing status messages to stdout.

4.7.3 Function Documentation

4.7.3.1 create_listening_socket()

```
SOCKET create_listening_socket (
            const char * host,
            const char * port)
```

Creates and binds a socket for a server to listen on.

Parameters

host	The local address to bind to (use NULL or 0 for any available interface).
port	The port number or service name (e.g., "8080").

Returns

A bound socket ready for listening.

4.7.3.2 drop_client()

```
void drop_client (
            struct client_info ** client_list,
            struct client_info * client)
```

Removes the given client from the server's client list of connections. Also frees memory used by client.

Parameters

client_list	A pointer to the head item in the client linked-list.
client	The client connection to be removed and shutdown.

4.7.3.3 get_client_address()

```
const char * get_client_address (
            struct client_info * info)
```

Retrieves the human-readable IP address string for a client.

Parameters

info	A pointer to the client_info struct.

Returns

A constant pointer to the client's IP address string.

4.7.3.4 get_client_info()

```
struct client_info * get_client_info (
            struct client_info ** client_list,
            SOCKET sock)
```

Searches the linked list for a client associated with a specific socket.

Parameters

client_list	A pointer to the head of the client linked list.
sock	The socket descriptor to search for.

Returns

A pointer to the client_info struct if found; NULL otherwise.

4.7.3.5 get_content_type()

```
const char * get_content_type (
            const char * path)
```

Returns the content-type value appropriate for the path arg's extension type.

Parameters

path	A valid path string to some file.

Returns

The appropriate content-header value for the path's extension.

4.7.3.6 print_socket_info()

```
void print_socket_info (
            const SOCKET sock)
```

Prints information about a network socket, given a bare socket as input.

Parameters

sock	The socket.

4.7.3.7 send_400()

```
void send_400 (
            struct client_info ** client_list,
            struct client_info * client)
```

Sends an HTTP 400 Bad Request response and closes the connection.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client that sent the malformed request.

4.7.3.8 send_404()

```
void send_404 (
            struct client_info ** client_list,
            struct client_info * client)
```

Sends an HTTP 404 Not Found response and closes the connection.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client that requested a non-existent resource.

4.7.3.9 serve_resource()

```
void serve_resource (
            struct client_info ** client_list,
            struct client_info * client,
            const char * path)
```

Locates a requested file on disk and transmits it to the client.

Parameters

client_list	A pointer to the head pointer of the client list.
client	The client requesting the resource.
path	The local filesystem path to the requested file.

4.7.3.10 start_winsock()

```
void start_winsock (
            WORD wVersion,
            LPWSADATA lpWSADATA)
```

Starts the Winsock API for the app.

Parameters

wVersion	The version of Winsock to be requested.
lpWSADATA	A structure holding the Winsock module's information.

4.7.3.11 wait_for_network_event()

```
fd_set wait_for_network_event (
            struct client_info ** client_list,
            SOCKET server)
```

Waits until a client's socket is ready to be handled.

Parameters

client_list	A pointer to the head of the connected clients list.
server	The listening server socket to monitor for new connections.

Returns

An fd_set containing all sockets that are ready for an I/O operation.

4.7.4 Variable Documentation

4.7.4.1 winsock_version

```
WORD winsock_version  [extern]
```

4.8 network_utils.h

Go to the documentation of this file.

```
00001
00007
00008 #pragma once
00009
00010 #include "winhders.h"
00011
00012 //--------------------------------------------------------------------------
00013 // Definitions
00014 //--------------------------------------------------------------------------
00015
00017 #define MAX_REQUEST_SIZE 2047
00018
```

```
00019 extern WORD winsock_version;
00020
00022 struct client_info {
00023     int address_length;
00024     struct sockaddr_storage address;
00025     char address_buffer[128];
00026     SOCKET socket;
00027     char request[MAX_REQUEST_SIZE + 1];
00028     int received;
00029     struct client_info *next;
00030 };
00031
00033 #define STATUS_LOG(fmt, ...) do { \
00034     fprintf(stdout, fmt "\n" __VA_OPT__(,) __VA_ARGS__); \
00035     fflush(stdout); \
00036 } while(0)
00037
00038 //------------------------------------------------------------------------------
00039
00040
00041 //------------------------------------------------------------------------------
00042 // Function Declarations
00043 //------------------------------------------------------------------------------
00044
00048 void start_winsock(WORD wVersion, LPWSADATA lpWSADATA);
00049
00053 const char *get_content_type(const char *path);
00054
00055
00056
00061 SOCKET create_listening_socket(const char *host, const char *port);
00062
00065 void print_socket_info(const SOCKET sock);
00066
00071 struct client_info *get_client_info(struct client_info **client_list, SOCKET sock);
00072
00073
00074
00079 void drop_client(struct client_info **client_list, struct client_info *client);
00080
00081
00082
00086 const char *get_client_address(struct client_info *info);
00087
00088
00089
00094 fd_set wait_for_network_event(struct client_info **client_list, SOCKET server);
00095
00096
00100 void send_400(struct client_info **client_list, struct client_info *client);
00101
00102
00103
00107 void send_404(struct client_info **client_list, struct client_info *client);
00108
00109
00110
00115 void serve_resource(struct client_info **client_list,
00116     struct client_info *client, const char *path);
00117
00118
00119 //------------------------------------------------------------------------------
```

4.9 winhders.h File Reference

```
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <direct.h>
```

Macros

- #define WIN32_LEAN_AND_MEAN

4.9.1 Macro Definition Documentation

4.9.1.1 WIN32_LEAN_AND_MEAN

```
#define WIN32_LEAN_AND_MEAN
```

4.10 winhders.h

Go to the documentation of this file.

```
00001
00007
00008 #pragma once
00009
00010 #ifndef WIN32_LEAN_AND_MEAN
00011 #define WIN32_LEAN_AND_MEAN
00012 #endif
00013
00014 #include <winsock2.h>
00015 #include <ws2tcpip.h>
00016 #include <windows.h>
00017 #include <direct.h>
00018
00019
00020 // For use with microsoft IDE/Tools.
00021 #pragma comment(lib, "Ws2_32.lib")
00022
```

Index